

به نام خدا



# برنامه سازی پیشرفته

دانشگاه شهید بهشتی - دانشکده مهندسی و علوم کامپیوتر

دکتر مجتبی وحیدی اصل

## وراثت

پارسا حمزه ئی

# فهرست مطالب

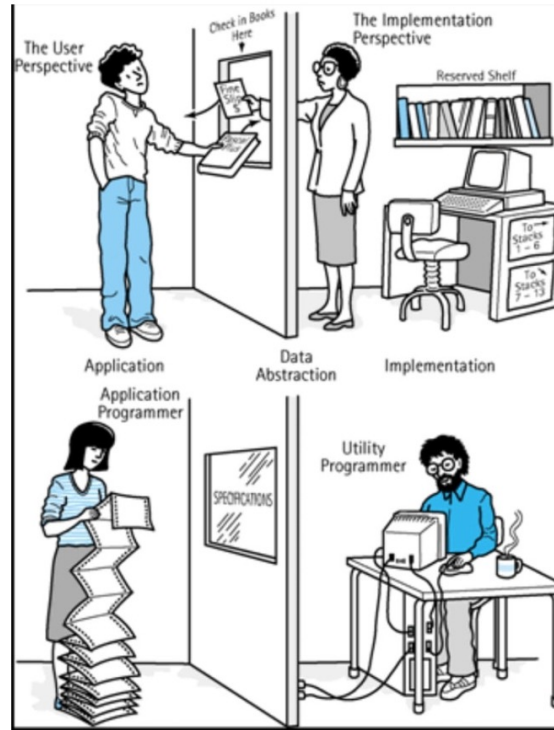
1. انتزاع و کپسوله بندی
2. پلی مورفیسم
3. ارثبری
4. انواع ارث بری
5. زیرکلاس ها و ابرکلاس ها
6. کلمه کلیدی SUPER

# انتزاع (Abstraction)

## انتزاع چیست؟

یکی از مزایای شیء‌گرایی انتزاع است، به پنهان کردن جزئیات داخلی از چشم کاربران، **انتزاع (abstraction)** گفته می‌شود.

کاربر باید تنها کارکرد و خروجی وسیله را مطابق با نیازهای خودش بداند.





#### نکته:

در جاوا برای رسیدن به انتزاع، از کلاس **abstract** و **interface** استفاده می‌کنیم.

#### Abstract Method چیست؟

متدی که برای همه اشیاء یک کلاس وجود دارد اما جزئیات دقیق و پیاده‌سازی آن، در خود کلاس غیر ممکن بوده و در هر زیرکلاس پیاده‌سازی می‌شود را متد انتزاعی می‌گویند.

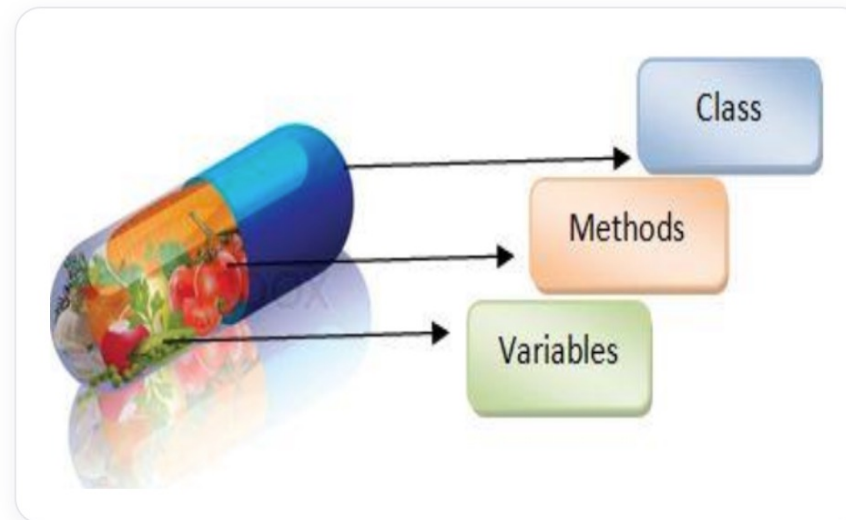
#### نکته:

اگر در یک کلاس، متدی انتزاعی تعریف کنیم، باید آن کلاس را انتزاعی تعریف کنیم! همچنین از یک کلاس انتزاعی، نمی‌توان نمونه‌ای ایجاد کرد!

# کپسوله بندی (encapsulation)

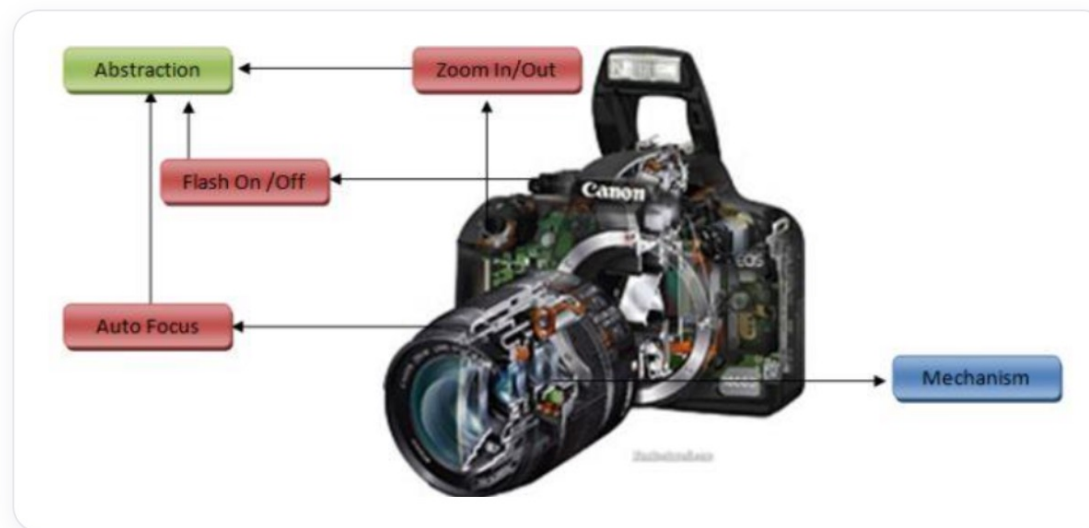
## کپسوله بندی چیست؟

ترکیب و بسته بندی داده ها و متدها در یک واحد به نام شیء، اصطلاحاً کپسوله بندی گفته می شود. به عنوان مثال هر کلاس در جاوا، یک کپسوله بندی است.



# تفاوت کپسوله‌بندی و انتزاع

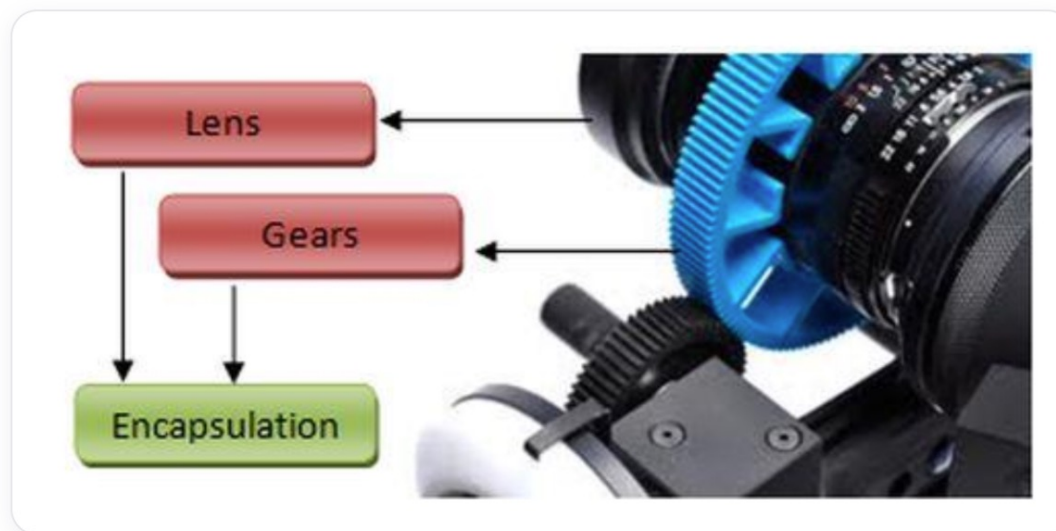
یک دوربین عکاسی دیجیتال مانند تصویر زیر در نظر بگیرید:



بر روی دفترچه راهنمای این دوربین نوشته شده است:  
"کاربر محترم، در هنگام استفاده از دوربین دیجیتال کافیست بر روی دکمه‌های zoom in و zoom out کلیک کنید و در هنگام زوم‌شدن دوربین حرکت لنز را حس خواهید نمود."

حال اگر دوربین را باز کنید با مکانیزم پیچیده آن روبه‌رو خواهید شد که برای شما قابل فهم نخواهد بود!  
فشردن دکمه عکاسی و گرفتن عکس (نتیجه دلخواه) **انتزاع** نامیده می‌شود.

همانطور که در تصویر زیر مشخص است زمانی که ما بر روی دکمه zoom in/out کلیک می‌کنیم، در درون ساختار دوربین مکانیزم‌هایی شامل چرخ‌دنده‌ها و لنزها، این خواسته ما را برآورده می‌کنند:



به ترکیب این چرخ‌دنده‌ها و لنزها اصطلاحاً **کپسوله‌بندی** گفته می‌شود که به عکاس امکان می‌دهد زومینگ را به نرمی و سادگی انجام دهد.

**پس میتوان نتیجه گرفت:** انتزاع از طریق کپسوله‌بندی امکان‌پذیر شده است.  
درواقع انتزاع جنبه طراحی مسئله را انجام می‌دهد و کپسوله‌بندی مسئول پیاده‌سازی مسئله است.

# چندریختی (Polymorphism)

## پلی مورفیسم یا چندریختی چیست؟

پلی مورفیسم یعنی کاری به شکل‌ها یا شیوه‌های مختلفی انجام شود. به عنوان مثال "صحبت کردن" در بین همه حیوانات مشترک است، اما هر حیوانی گویش و صدای خاص خودش را دارد:



در جاوا برای رسیدن به پلی مورفیسم از باز نویسی متد (overriding) استفاده می‌کنیم.

**نکته:** در این مثال متد "صحبت کردن" برای کلاس حیوان انتزاعی **abstract** می‌باشد.



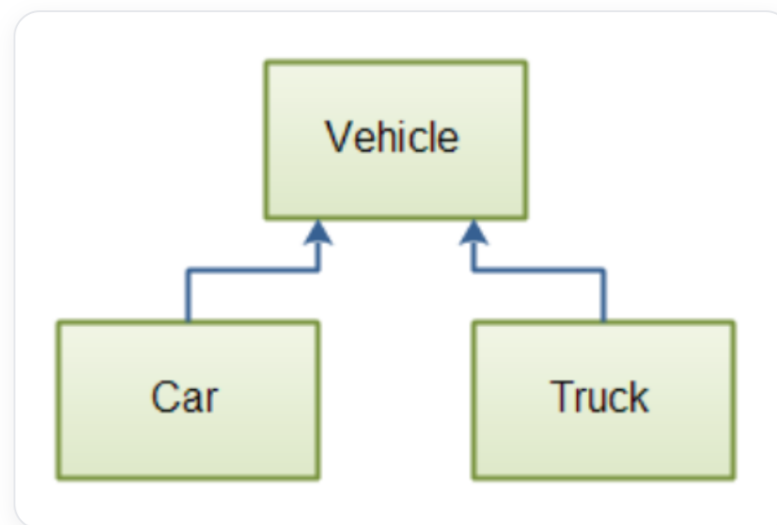
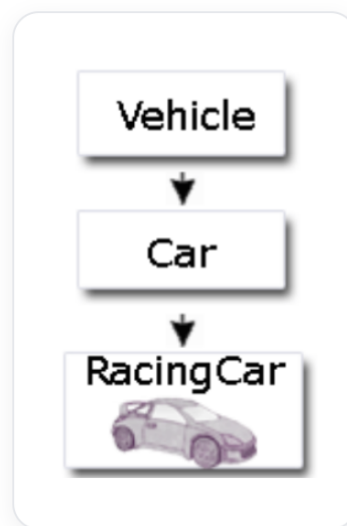
# ارثبری (inheritance)

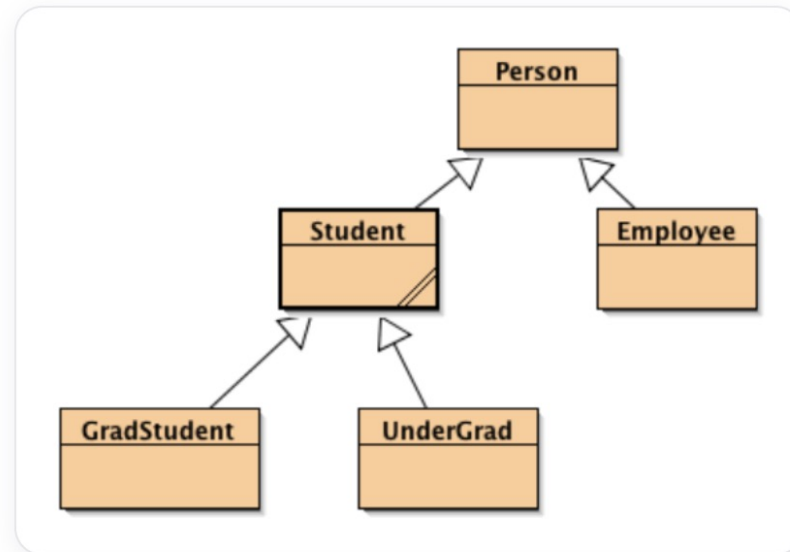
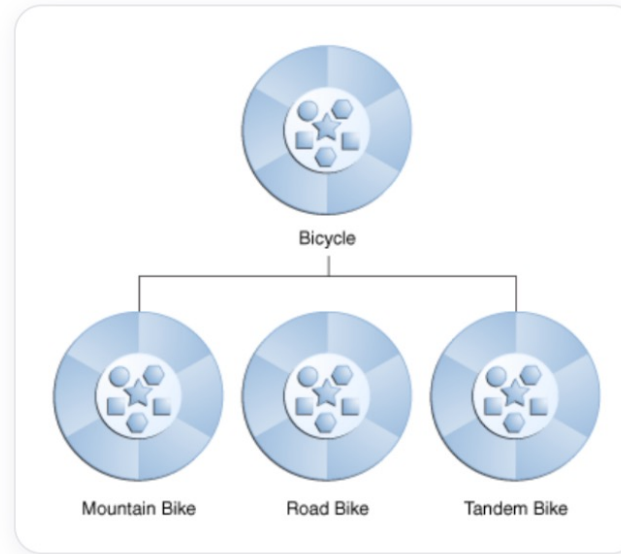
## ارثبری چیست؟

فرض کنید می‌خواهید کلاس‌هایی تعریف کنید تا شکل‌هایی مثل دایره، مثلث و مستطیل را مدلسازی کند. این کلاس‌ها دارای صفات مشترک زیادی هستند مانند مساحت، محیط و ...

به همین دلیل بهترین راه برای طراحی این کلاس‌ها و پرهیز از تکرار نویسی، استفاده از **وراثت** است!

مثالهایی از وراثت در شکل‌های زیر مشاهده می‌کنید:

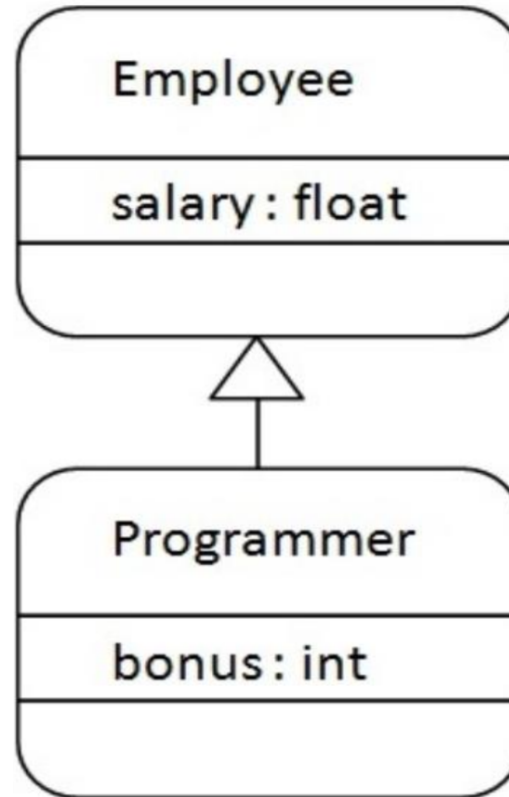




قاعده نحوی وراثت به صورت زیر است:

```
class Subclass-name extends Superclass-name {  
    //methods and fields  
}
```

کلمه کلیدی **extends** می‌گوید که شما کلاس جدیدی ایجاد کرده‌اید که از یک کلاس موجود ارث‌بری می‌کند. به کلاس موجود، **ابركلاس** یا کلاس والد (پدر) گفته می‌شود و به کلاس جدید نیز، **کلاس فرزند** می‌گویند. در تساوی‌ر، پیکان همیشه از فرزند به پدر رسم می‌شود:



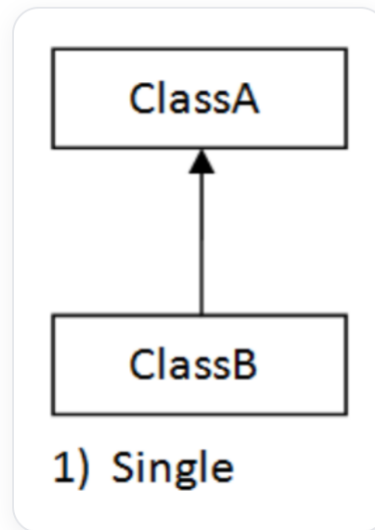
بر اساس شکل بالا، برنامه‌ای بنویسید که حقوق (salary) و پاداش (bonus) یک برنامه‌نویس که از کارمند ارث‌بری می‌کند، چاپ کند.

```
class Employee {  
    float salary = 40000 ;  
}  
  
class Programmer extends Employee {  
    int bonus = 10000 ;  
    public static void main(String args[]){  
        Programmer p = new Programmer();  
        System.out.println("Programmer salary is : " + p.salary);  
        System.out.println("Bonus of Programmer is : " + p.bonus);  
    }  
}
```

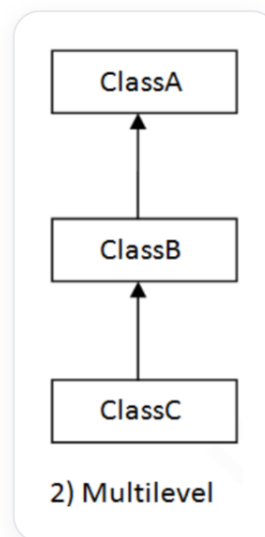
# انواع ارثبری

سه نوع ارثبری بر اساس کلاس والد قابل تعریف است:

- ساده:

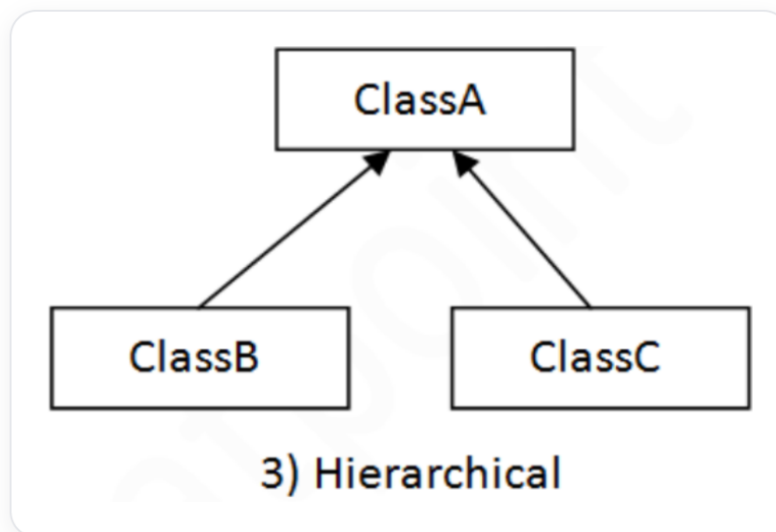


• چند سطحی:





- سلسله مراتبی:

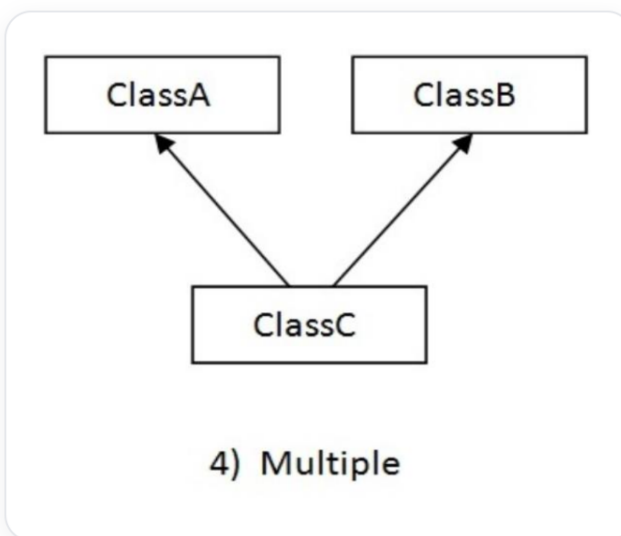


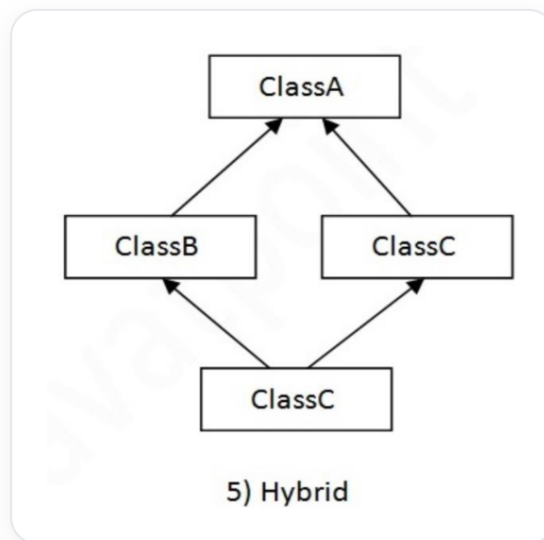
**نکته:** در جاوا ارثبری ترکیبی (هیبرید) و چندگانه تنها از طریق کلاس‌های واسط امکان‌پذیر است که بعداً توضیح داده خواهد شد.

# ارثبری چندگانه

ارثبری چندگانه چیست؟

هرگاه کلاسی بخواهد از چند کلاس ارثبری کند، به این نوع ارثبری، ارثبری چندگانه گفته می‌شود.





در جاوا ارثبری چندگانه پشتیبانی نمیشود. دلیل این امر، کاهش پیچیدگی و ساده‌سازی زبان است!

## مثالی از ارثبری چندگانه:

در مثال زیر، دو کلاس والد متدی به نام msg دارند و کامپایلر نمی‌داند کدام را برای فرزند ارثبری کند.

لذا در هر حالت (حتی عدم وجود متدهای هم‌نام در کلاس‌های والد) جاوا برای ارثبری چندگانه، خطای زمان کامپایل می‌گیرد.

```
class A{
    void msg(){
        System.out.println("hello");
    }
}
class B{
    void msg(){
        System.out.println("welcome");
    }
}
class C extends A,B { //suppose if it were
    public static void main(String args[]){
        C obj = new C();
        obj.msg(); //Now wich msg() method be invoked?
    }
}
```

# Up/Down Casting

هر زیرکلاس، که از ابرکلاس ارثبری می‌کند، از نوع ابرکلاس نیز می‌باشد.  
به عنوان مثال دایره و مستطیل و... هر کدام یک شکل هستند، یا گربه و سگ و ... هر کدام حیوان هستند!  
به همین دلیل می‌توان زیرکلاس‌ها را از نوع ابرکلاس هم تعریف کرد و آن‌ها را به زیرکلاس (اگر از جنس زیرکلاس باشند) تبدیل کرد.  
البته دقت کنید در متدهای انتزاعی، نوع دقیق شیء تعیین کننده رفتار است، نه نوع ارجاع آن!

```
class Animal{  
}  
  
class Dog extends Animal {  
    public static void main(String args[]){  
        Animal a = new dog();  
        Dog b = (Dog) a;  
    }  
}
```

# تمرین زیرکلاس و ابرکلاس

برنامه‌ای با مشخصات زیر بنویسید:

**1 class name:**

GeometricObject

**fields:**

- color: String
  - ▶ The color of the object(default: white)
- filled: boolean
  - ▶ Indicates whether the object is filled with a color(default: false)
- dateCreated: java.util.Date
  - ▶ The date when the object was created

**methods:**

- + GeometricObject()
  - ▶ Creates a GeometricObject.
- + GeometricObject(color: String , filled: boolean)
  - ▶ Creates a GeometricObject with the specified color and filled values.
- + getColor(): String
  - ▶ Returns the filled property.
- + setColor(color: String): void
  - ▶ Sets a new color.
- + isFilled(filled: boolean): void
  - ▶ Returns the filled property.
- + setFilled(filled: boolean): void
  - ▶ set a new filled property.
- + getDateCreated(): java.util.Date
  - ▶ Returns the dateCreated.
- + toString(): String

2 **class name:**

Circle

**fields:**

– radius: double

**methods:**

+ Circle()

+ Circle(radius: double)

+ Circle(radius: double , color: String , filled: boolean)

+ getRadius(): double

+ setRadius(radius: double): void

+ getArea(): double

+ getPerimeter(): double

+ getDiameter(): double

+ printCircle(): void

---



3

**class name:**

Rectangle

**fields:**

- width: double
- height: double

**methods:**

- + Rectangle()
- + Rectangle(width: double , height ; double)
- + Rectangle(width: double , height ; double , color: String , filled: boolean)
- + getWidth(): double
- + setWidth(width: double): void
- + getHeight(): double
- + setHeight(height: double): void
- + getArea(): double
- + getPerimeter(): double

```
public class GeometricObject1 {
    private String color = "white";
    private boolean filled;
    private java.util.Date dateCreated;
    public GeometricObject1() {
        dateCreated = new java.util.Date();
    }
    public GeometricObject1(String Color, boolean filled) {
        dateCreated = new java.util.Date();
        this.color = color;
        this.filled = filled;
    }
    public String getColor() {
        return color;
    }
    public void setColor(String color) {
        this.color = color;
    }
    public boolean isFilled() {
        return filled;
    }
    public void setFilled(boolean filled) {
        this.filled = filled;
    }
}
```

```
    public java.util.Date getDateCreated() {
        return dateCreated;
    }
    public String toString() {
        return "created on " + dateCreated + "\n" + "color: " +
            color + " and filled: " + filled;
    }
}
```

```

public class Circle4 extends GeometricObject1 {
    private double radius;
    public Circle4() {
    }
    public Circle4(double radius) {
        super();
        this.radius = radius;
    }
    public Circle4(double radius, String color, boolean filled) {
        super(color, filled);
        this.radius = radius;
        //setColor(color);
        //setFilled(filled);
    }
    public double getRadius() {
        return radius;
    }
    public void setRadius(double radius) {
        this.radius = radius;
    }
    public double getArea() {
        return radius * radius * Math.PI;
    }
    public double getDiameter() {
        return 2 * radius;
    }
}

```

```

    public double getPerimeter() {
        return 2 * radius * Math.PI;
    }
    public void printCircle() {
        System.out.println(toString() + "The circle is
        created " + getDateCreated() +
        " and the radius is " + radius);
    }
    public String toString() {
        return "Circle WWW " + getColor() +
        super.toString();
    }
}

```

```
public class Rectangle1 extends GeometricObject1 {
    private double width;
    private double height;
    public Rectangle1() {
    }
    public Rectangle1(double width, double height) {
        this.width = width;
        this.height = height;
    }
    public Rectangle1(double width, double height, String color, boolean filled) {
        this.width = width;
        this.height = height;
        setColor(color);
        setFilled(filled);
    }
    public double getWidth() {
        return width;
    }
    public void setWidth(double width) {
        this.width = width;
    }
    public double getHeight() {
        return height;
    }
}
```

```
    public void setHeight(double height) {
        this.height = height;
    }
    public double getArea() {
        return width * height;
    }
    public double getPerimeter() {
        return 2 * (width + height);
    }
}
```

```
public class TestCircleRectangle {  
    public static void main(String[] args) {  
        Circle4 circle = new Circle4(1);  
        System.out.println("A circle " + circle.toString());  
        System.out.println("The radius is " + circle.getRadius());  
        System.out.println("The area is " + circle.getArea());  
        System.out.println("The diameter is " + circle.getDiameter());  
        Rectangle1 rectangle = new Rectangle1(2, 4);  
        System.out.println("\nA rectangle " + rectangle.toString());  
        System.out.println("The area is " + rectangle.getArea());  
        System.out.println("The perimeter is " + rectangle.getPerimeter());  
    }  
}
```

# ارثبری از ابرکلاس‌ها

آیا سازنده ابرکلاس‌ها، ارثبری می‌شوند؟

خیر به ارث برده نمی‌شوند!  
قبل‌تر دیدیم که سازنده‌ها به صورت ضمنی یا صریح در بدنه کلاس قرار داده می‌شوند.

---

یک سازنده برای ایجاد یک نمونه (یک شیء جدید) از یک کلاس استفاده می‌شود.  
برخلاف فیلدهای داده‌ای و متدها، با استفاده از کلمه کلیدی **super** انجام می‌شود. اگر کلمه کلیدی **super** استفاده نشود سازنده no-arg کلاس پدر به طور خودکار فراخوانی می‌شود.

درواقع یک سازنده ممکن است یک سازنده سربار گذاری شده یا سازنده ابرکلاسش را فراخوانی کند.  
اگر هیچ یک از این دو مشخصا فراخوانی نشوند، کامپایلر به طور خودکار **super()** را به عنوان اولین دستور در سازنده قرار می دهد.  
به عنوان مثال کد زیر:

```
public A(){  
}
```

به صورت زیر کامپایل می شود:

```
public A(){  
    super();  
}
```

---

همچنین کد زیر:

```
public A(double d){  
    // some statements  
}
```

به صورت زیر کامپایل می شود:

```
public A(){  
    super();  
    // some statements  
}
```



## کلمه کلیدی super:

کلمه کلیدی **super** در کلاس فرزند استفاده می‌شود و به والد آن کلاس اشاره می‌کند. اگر از کلمه کلیدی **super** برای ارجاع به متغیر نمونه کلاس والد، وقتی هر دو یک فیلد هم‌نام دارند، استفاده نشود:

```
class Vehicle{
    int speed = 50;
}

class Bike3 extends Vehicle{
    int speed = 100;

    void display(){
        System.out.println(speed); // will print speed of Bike
    }
}

public static void main(String args[]){
    Bike3 b = new Bike3();
    b.display();
}
```

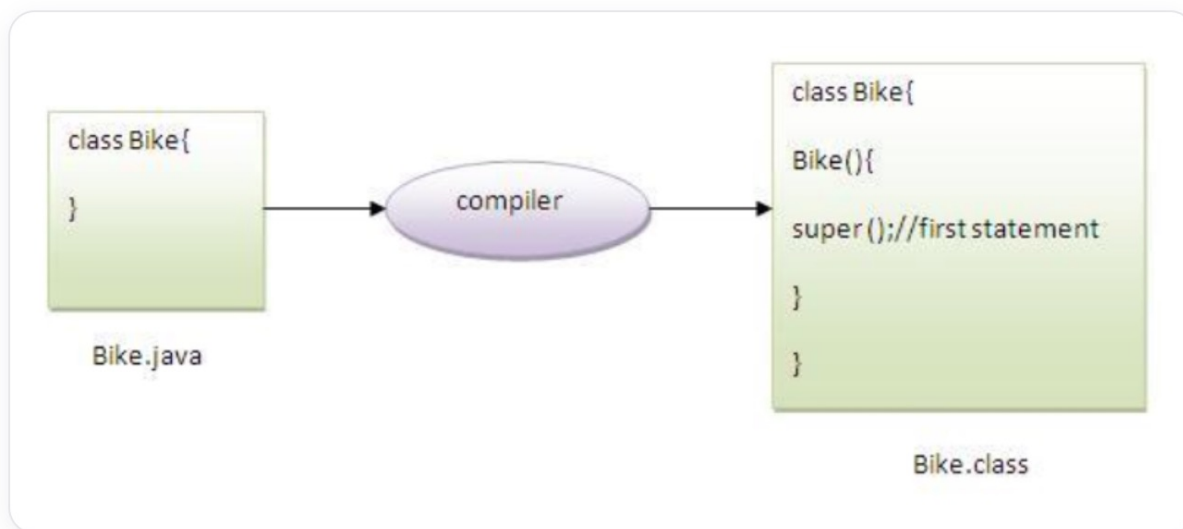
```
class Vehicle{
    Vehicle(){
        System.out.println("Vehicle is created!");
    }
}

class Bike5 extends Vehicle{
    Bike5(){
        super(); //will invoke parent class constructor
        System.out.println("Bike is created");
    }
}

public static void main(String args[]){
    Bike5 b = new Bike5();
}
```

## کلمه کلیدی super برای فراخوانی متد کلاس والد

همانطور که می‌دانیم اگر خود ما سازنده‌ای قرار ندهیم، سازنده پیش فرض توسط کامپایلر ایجاد می‌شود. در کلاس فرزند، کامپایلر به طور خودکار کلمه `super()` را به عنوان اولین دستور به سازنده فرزند اضافه می‌کند.



در مواقعی که متد فرزند با متد والد هم نام باشد، از کلمه کلیدی `super` برای فراخوانی متد کلاس والد استفاده می شود:

```
class Person{
    void message(){
        System.out.println("Welcome");
    }
}

class Student16 extends Person{
    void message(){
        System.out.println("Welcome to java");
    }
}

void display(){
    message() //will invoke current class message() method
    super.message() //will invoke parent class message() method
}

public static void main(String args[]){
    Student16 s = new Student16();
    s.display();
}
```

# زنجیره سازنده‌ها

ایجاد یک شیء از یک کلاس تمامی سازنده‌های آن و ابرکلاسهای آن را در قالب زنجیره‌ای وراثتی فراخوانی می‌کند که به آن زنجیره سازنده‌ها گفته می‌شود.

به عنوان مثال کد زیر را در نظر بگیرید:

```
public class Faculty extends Employee {
    public static void main(String[] args) {
        new Faculty();
    }
    public Faculty() {
        System.out.println("(4) Faculty's no-arg constructor is invoked");
    }
}

class Employee extends Person {
    public Employee() {
        this("(2) Invoke Employee's overloaded constructor");
        System.out.println("(3) Employee's no-arg constructor is invoked");
    }
    public Employee(String s) {
        System.out.println(s);
    }
}

class Person {
    public Person() {
        System.out.println("(1) Person's no-arg constructor is invoked");
    }
}
```

# ردگیری اجرای کد

در هنگام اجرای کد، به ترتیب زیر اجرا می‌شود:

1\_ Start from the main method:

```
public static void main(String args[])
```

در ابتدا خط دوم، تابع `main` اجرا می‌شود.

---

2\_ Invoke Faculty constructor:

```
new Faculty();
```

سپس خط سوم، یعنی متد `Faculty` اجرا می‌شود که ارجاع می‌شود به `public Faculty()`.

---

### 3\_ Invoke Employee's no-arg constructor:

```
public Employee();
```

در ادامه چون **faculty** از **employee** ارث بری می‌کند، سازنده **employee** اجرا می‌شود یعنی خط یازدهم.

---

### 4\_ Invoke Employee(String) constructor ;

```
this("(2) Invoke Employee's overloaded constructor");  
public Employee(String s);
```

خود سازنده بدون آرگومان، سازنده **Employee** با ورودی رشته را اجرا می‌کند.

---

```
5_ Invoke Person() constructor:  
    public Person();
```

چون **Employee** از کلاس **Person** ارث بری می‌کند، در ادامه سازنده بدون آرگومان **Person()** را صدا می‌زنند.  
در ادامه متدهای **println** به ترتیب زیر اجرا می‌شوند:

```
6_ System.out.println("(1) Person's no-arg constructor is invoked");  
7_ System.out.println(s);  
8_ System.out.println("(3) Employee's no-arg constructor is invoked");  
9_ System.out.println("(4) Faculty's no-arg constructor is invoked");
```

بعد اجرای متدهای پرینت، برنامه پایان می‌یابد.

**نکته:** اگر در یک ابرکلاس، سازنده‌ای غیر no-arg تعریف کنیم، باید در زیرکلاس‌ها سازنده super را با آرگومان صدا کنیم.

خطای برنامه زیر را بیابید:

```
public class Apple extends Fruit {  
}  
class Fruit {  
    public Fruit(String name) {  
        System.out.println("Fruit's constructor is invoked");  
    }  
}
```



# نکات اضافی

دونستن نکات زیر برای فهمیدن درس و نوشتن کد کمکتون می‌کنه (:

- polymorphism و Encapsulation و Inheritance از ویژگی‌های مهم زبان های شیء‌گرا می‌باشند.
- اگر کلاسی دارای متد انتزاعی باشد (حتی یک متد)، آن کلاس باید انتزاعی **abstract** تعریف شود.
- اگر کلاسی از یک کلاس انتزاعی ارث‌بری کند، باید تمام متدهای انتزاعی را کامل پیاده‌سازی کند در غیر این صورت باید آن هم انتزاعی باشد.
- در جاوا 8 به نوعی وراثت چندگانه ممکن شد، راجب آن تحقیق کنید.

# خودمون رو بسنجیم

این بخش برای این طراحی شده که در پایان مطالعه این اسلاید، بتونی خودت رو محک بزنی و ببینی آیا مفاهیم رو به خوبی یاد گرفتی یا نه. سوالات زیر رو مرور کن و سعی کن بدون نگاه کردن به متن درس، به اون ها پاسخ بدی.

- انتزاع یا **abstraction** چیست و چه تفاوتی با کپسوله‌بندی **encapsulation** دارد؟
- برای وراثت یک مثال بزنی و انواع آن را نام ببرید.
- کلمه کلیدی **super** چه کاربرد و عملکردی دارد؟

## پایان

در صورت هرگونه سوال یا پیشنهاد میتونید با من  
در ارتباط باشید:

gmail: Parsahamzeie@gmail.com  
telegram: @ParsaHami